

20 - Gateway 与生产部署

阅读时间：约 2 小时

前置知识：06 - 安装与上手、18 - WebUI 实战

学习目标：掌握 Nanobot Gateway 的架构、启动流程、多实例部署、Docker 部署

目录

- 20.1 Gateway 概述
 - 20.2 Gateway 启动流程
 - 20.3 Gateway 核心服务
 - 20.4 多实例部署
 - 20.5 Docker 部署
 - 20.6 Linux Service 部署
 - 20.7 面试话术
-

20.1 Gateway 概述

Gateway 是 Nanobot 的生产级运行模式，相比 CLI 的 `nanobot agent`，Gateway 提供：- 多 Channel 并发支持 - 内置 WebUI - 后台系统任务（Dream、Heartbeat）- 持久化会话管理 - Health Check 端点

启动命令：

```
nanobot gateway
```

20.2 Gateway 启动流程

Gateway 启动时按顺序初始化以下服务：

1. 加载配置（`~/nanobot/config.json`）
↓
 2. 初始化 Provider（LLM 后端）
↓
 3. 注册 Channel（Telegram、Discord、WebSocket等）
↓
 4. 启动 MessageBus（消息队列）
↓
 5. 启动 AgentLoop（Agent 循环）
↓
 6. 注册系统任务：
 - Dream 记忆摘要
 - Heartbeat 心跳检查
 - Cron 定时任务
↓
 7. 启动 Health endpoint（默认端口 18790）
↓
 8. 启动 WebSocket channel（WebUI，默认端口 8765）
↓
 9. 开始接收消息
-

20.3 Gateway 核心服务

20.3.1 Chat Channels

Gateway 可以同时运行多个 Channel:

```
{
  "channels": {
    "telegram": { "enabled": true, "token": "${TELEGRAM_TOKEN}" },
    "discord": { "enabled": true, "token": "${DISCORD_TOKEN}" },
    "feishu": { "enabled": true, "appId": "${FEISHU_APP_ID}" },
    "websocket": { "enabled": true }
  }
}
```

20.3.2 WebSocket Channel (WebUI)

- 端口: 8765 (默认)
- 协议: WebSocket
- 功能: 实时聊天、文件编辑、Model switching

20.3.3 Dream 系统任务

- 自动处理未读会话历史
- 生成智能摘要更新到 MEMORY.md
- 通过 .dream_cursor 游标跟踪进度

20.3.4 Heartbeat 心跳任务

```
{
  "gateway": {
    "heartbeat": {
      "enabled": true,
      "interval": 1800
    }
  }
}
```

- 定期检查系统健康状态
- 默认间隔: 1800 秒 (30 分钟)
- 存储位置: <workspace>/cron/jobs.json

20.3.5 Cron 定时任务

支持用户自定义定时任务: - 基于 cron 表达式 - 自动触发 Agent 对话 - 持久化存储

20.3.6 Health Endpoint

- 默认端口: 18790
- URL: http://127.0.0.1:18790/health
- 用途: Docker health check、监控系统

20.4 多实例部署

场景: 运行多个独立的 Bot

实例 1: 个人助手

nanobot gateway --config ~/.nanobot/config-personal.json --workspace ~/workspaces/personal

实例 2: 工作助手

```
nanobot gateway --config ~/.nanobot/config-work.json --workspace ~/workspaces/work
```

实例 3: 客服 Bot

```
nanobot gateway --config ~/.nanobot/config-support.json --workspace ~/workspaces/support
```

配置隔离

每个实例需要: - 独立的配置文件 - 独立的 workspace 目录 - 独立的 Gateway 端口 (如果在同一机器)

```
{
  "gateway": {
    "port": 18791
  },
  "channels": {
    "websocket": {
      "port": 8766
    }
  }
}
```

20.5 Docker 部署

Dockerfile 示例

```
FROM python:3.12-slim
```

```
WORKDIR /app
```

```
# 安装 nanobot
```

```
RUN pip install nanobot-ai
```

```
# 复制配置
```

```
COPY config.json /root/.nanobot/config.json
```

```
# 暴露端口
```

```
EXPOSE 8765 18790
```

```
# 启动 Gateway
```

```
CMD ["nanobot", "gateway"]
```

docker-compose.yml

```
version: '3.8'
```

```
services:
```

```
  nanobot:
```

```
    build: .
```

```
    ports:
```

```
      - "8765:8765" # WebUI
      - "18790:18790" # Health
```

```
    volumes:
```

```
      - ./workspace:/root/workspace
      - ./config.json:/root/.nanobot/config.json
```

```
    environment:
```

```
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - TELEGRAM_TOKEN=${TELEGRAM_TOKEN}
```

```
restart: unless-stopped
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:18790/health"]
  interval: 30s
  timeout: 10s
  retries: 3
```

启动

```
docker-compose up -d
```

20.6 Linux Service 部署

systemd service 文件

创建 /etc/systemd/system/nanobot.service:

```
[Unit]
Description=Nanobot AI Gateway
After=network.target

[Service]
Type=simple
User=nanobot
Group=nanobot
WorkingDirectory=/opt/nanobot
ExecStart=/opt/nanobot/venv/bin/nanobot gateway
Restart=always
RestartSec=10
Environment=PATH=/opt/nanobot/venv/bin
EnvironmentFile=/opt/nanobot/.env

[Install]
WantedBy=multi-user.target
```

环境变量文件

创建 /opt/nanobot/.env:

```
OPENAI_API_KEY=sk-xxx
TELEGRAM_TOKEN=123456:ABC-xxx
```

启动服务

```
# 重载 systemd
sudo systemctl daemon-reload
```

```
# 启动服务
sudo systemctl start nanobot
```

```
# 设置开机自启
sudo systemctl enable nanobot
```

```
# 查看状态
sudo systemctl status nanobot
```

```
# 查看日志
sudo journalctl -u nanobot -f
```

20.7 面试话术

问题：Nanobot Gateway 的架构是怎样的？

“Nanobot Gateway 是生产级运行模式，启动时会按顺序初始化多个服务：首先加载配置和 Provider，然后注册所有启用的 Channel（Telegram、Discord、WebSocket 等），启动 MessageBus 和 AgentLoop，最后注册系统任务（Dream 记忆摘要、Heartbeat 心跳检查、Cron 定时任务）并开放 Health endpoint（18790 端口）和 WebUI（8765 端口）。Gateway 支持多实例部署，每个实例通过独立的配置文件和 workspace 实现隔离。”

问题：如何实现 Nanobot 的高可用部署？

“可以通过 Docker Compose 或 systemd 实现高可用部署。Docker 方案使用 docker-compose.yml 定义服务，配置 health check 自动监控 Gateway 状态，设置 restart policy 自动重启失败容器。systemd 方案创建 service 文件，配置 Restart=always 和 EnvironmentFile 管理环境变量。多实例部署时，每个实例使用独立的配置文件、workspace 和端口，通过 -config 和 -workspace 参数指定。”

扩展阅读：18 - WebUI 实战 ——WebUI 配置和使用